

Título

Bernardo Maia Coelho¹, Eduardo Fontoura Costa², Luiz Henrique Romero³
 ICMC/USP, São Carlos, SP

1 Formulação do Problema

Dada uma função $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$, buscamos uma aproximação por combinação linear de funções base $\Phi = \{\varphi_1, \varphi_2, \dots, \varphi_k\} \subset V$, onde V é o espaço vetorial das funções f dotado do produto interno

$$\langle f, g \rangle = \int_{\Omega} f(\vec{x}) \cdot g(\vec{x}) d\vec{x},$$

com $\cdot$ denotando o produto escalar usual em $\mathbb{R}^m$ e $\Omega \subset \mathbb{R}^n$ um domínio limitado e mensurável. Para que esse produto interno esteja bem definido e seja finito, restringimos nossa atenção às funções cujos componentes são quadrado-integráveis sobre o domínio Ω . Isso nos leva ao subespaço

$$V_{\Omega} = \{f \in V \mid f_i \in L^2(\Omega) \text{ para } i = 1, \dots, m\},$$

o qual, quando dotado do produto interno acima, constitui um espaço de Hilbert[2].

A linearidade do produto escalar em $\mathbb{R}^m$ permite reescrever o produto interno entre $f, g \in V_{\Omega}$ como

$$\langle f, g \rangle = \int_{\Omega} \sum_{i=1}^m f_i(\vec{x}) g_i(\vec{x}) d\vec{x} = \sum_{i=1}^m \int_{\Omega} f_i(\vec{x}) g_i(\vec{x}) d\vec{x}.$$

Em particular, se adotarmos um domínio $\Omega = [a_1, b_1] \times \dots \times [a_n, b_n] \subset \mathbb{R}^n$, a expressão acima assume a forma

$$\langle f(\vec{x}), g(\vec{x}) \rangle = \sum_{i=1}^m \int_{a_1}^{b_1} \dots \int_{a_n}^{b_n} f_i(x_1, \dots, x_n) g_i(x_1, \dots, x_n) dx_n \dots dx_1.$$

Quando a função f é conhecida apenas por meio de uma amostragem em um conjunto de pontos $\{p_j\}_{j=1}^N \subset \Omega$, é possível aproximar o produto interno pela soma discreta

$$\langle f, g \rangle \approx \sum_{i=1}^m \sum_{j=1}^N f_i(\vec{p}_j) g_i(\vec{p}_j).$$

Com isso, o problema pode ser formulado como a resolução de um sistema linear

$$G\vec{a} = \vec{b},$$

conhecido como sistema de equações normais dos mínimos quadrados[1, 2], em que $G \in \mathbb{R}^{k \times k}$ é a matriz de Gram, com entradas $G_{i,j} = \langle \varphi_i, \varphi_j \rangle$; $\vec{a} \in \mathbb{R}^k$ contém os coeficientes da combinação linear; e $\vec{b} \in \mathbb{R}^k$ possui entradas $b_i = \langle f, \varphi_i \rangle$.

¹autora1@email

²autora2@email

³autora1@email

2 Regressão Polinomial

Uma forma natural e eficaz de construir uma base é por meio de polinômios, sendo que o conjunto $\{\varphi_k\}$ pode ser formado por todas as funções da forma

$$\varphi_k = \hat{e}_d c_{d,\vec{\alpha}} x^{\vec{\alpha}},$$

onde o vetor de expoentes $\vec{\alpha} = (\alpha_1, \alpha_2, \dots, \alpha_n) \in \mathbb{N}^n$ varia sobre todas as combinações cuja soma é $|\vec{\alpha}| = \sum_{i=1}^n \alpha_i \leq r$ — com r sendo o grau máximo do polinômio —, e $\hat{e}_d \in \mathbb{R}^m$ é o vetor unitário com valor 1 na componente de índice d e zero nas demais. Portanto, o polinômio pode ser escrito como

$$P_{\leq r}(\vec{x}) = \sum_{d=1}^m \sum_{\substack{\vec{\alpha} \in \mathbb{N}^n \\ |\vec{\alpha}| \leq r}} c_{d,\vec{\alpha}} \hat{e}_d \vec{x}^{\vec{\alpha}}, \quad \vec{x}^{\vec{\alpha}} := \prod_{i=1}^n x_i^{\alpha_i}.$$

onde $\vec{x}^{\vec{\alpha}}$ denota o monômio associado ao multi-índice $\vec{\alpha}$, conforme [1]. Expandindo os termos homogêneos $x^{\vec{\alpha}}$, obtemos

$$P_{\leq r}(\vec{x}) = \sum_{d=1}^m \sum_{j=0}^r \sum_{\substack{\vec{\alpha} \in \mathbb{N}^n \\ |\vec{\alpha}|=j}} c_{d,\vec{\alpha}} \hat{e}_d \prod_{i=1}^n x_i^{\alpha_i}.$$

Note que aqui assumimos a existência de algum mapeamento razoável na forma

$$(d, \vec{\alpha}) \rightarrow k.$$

O índice k referencia cada função base de maneira unívoca e nos permite adotar a modelagem matricial usual: montamos o sistema linear $G\vec{a} = \vec{b}$ em que G é a matriz de Gram associada às $\{\varphi_k\}$.

$$\begin{bmatrix} \langle \varphi_0, \varphi_0 \rangle & \langle \varphi_0, \varphi_1 \rangle & \cdots & \langle \varphi_0, \varphi_n \rangle \\ \langle \varphi_1, \varphi_0 \rangle & \langle \varphi_1, \varphi_1 \rangle & \cdots & \langle \varphi_1, \varphi_n \rangle \\ \vdots & \vdots & \ddots & \vdots \\ \langle \varphi_n, \varphi_0 \rangle & \langle \varphi_n, \varphi_1 \rangle & \cdots & \langle \varphi_n, \varphi_n \rangle \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_n \end{bmatrix} = \begin{bmatrix} \langle f, \varphi_0 \rangle \\ \langle f, \varphi_1 \rangle \\ \vdots \\ \langle f, \varphi_n \rangle \end{bmatrix}$$

Essa formulação estabelece as relações que serão usadas nas seções seguintes para analisar o condicionamento numérico e as estratégias de solução.

3 Otimização da Regressão

Essa formulação já basta para viabilizar a modelagem computacional. Ainda assim, convém enxergar onde estão os gargalos antes de escrever uma linha de código. O sistema linear proposto tem alta complexidade, e vale a pena organizar o raciocínio desde já. Seja N_r o número de termos $\hat{e}_d \vec{x}^{\vec{\alpha}}$ de grau igual a r ($\|\vec{\alpha}\| = r$); então

$$N_j = m \cdot \#\{\vec{\alpha} \in \mathbb{N}^n : |\vec{\alpha}| = j\} = m \binom{n+j-1}{j}.$$

Assim, podemos calcular o número de termos existentes de grau até r

$$\|\{\varphi_k\}\| = N_{\leq r} = \sum_{j=0}^r N_j = m \sum_{j=0}^r \binom{n+j-1}{j} = m \binom{n+r}{r}.$$

Ou seja, a nossa matriz G tem dimensão considerável:

$$N_{\leq r} \times N_{\leq r} = \left(m \binom{n+r}{r}\right)^2 = m^2 \binom{n+r}{r}^2 = m^2 \binom{n+r}{n}^2 = m^2 \frac{(n+r)!^2}{(n!)^2 (r!)^2}.$$

Isso nos dá a complexidade para a regressão polinomial:

$$p = \binom{n+r}{r}.$$

Via método QR:

$$T_{\text{QR}}(N, n, r, m) = O(Np^2 + p^2 m)$$

Via equações normais (Cholesky):

$$T_{\text{NE}}(N, n, r, m) = O(Np^2 + p^3 + p^2 m)$$

Essas estimativas refletem a comparação clássica entre decomposição QR e equações normais/Cholesky na resolução de mínimos quadrados[1, 2].

Os fatoriais elevados a 2 ou 3 já são razão suficiente para deixar qualquer cientista da computação em alerta. Não conseguimos melhorar muito a complexidade assintótica desse algoritmo, porém é possível extrair um ganho de desempenho prático explorando uma propriedade estrutural de G . Note que, por definição,

$$\hat{e}_i \perp \hat{e}_j \quad i \neq j \quad \implies \langle \hat{e}_i c_{d,\vec{\alpha}} \vec{x}^{\vec{\alpha}}, \hat{e}_j c_{d,\vec{\beta}} \vec{x}^{\vec{\beta}} \rangle = 0$$

Então, com uma boa escolha de mapeamento do índice k , nossa matriz G torna-se diagonal por blocos:

$$A = \begin{matrix} & e_1 & e_2 & \cdots & e_n \\ \begin{matrix} e_1 \\ e_2 \\ \vdots \\ e_h \end{matrix} & \begin{bmatrix} A_1 & 0 & \cdots & 0 \\ 0 & A_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & A_h \end{bmatrix} \end{matrix},$$

Usando o delta de Kronecker, podemos resumir essa propriedade da seguinte forma:

$$G_{ij} = \langle \varphi_i, \varphi_j \rangle \cdot \delta_{i,j}, \quad \hat{e}_i \perp \hat{e}_j$$

Logo, em vez de resolver um sistema linear de tamanho catastrófico, exploramos as propriedades de matrizes diagonais por bloco e decomposomos o problema em m sistemas menores do tipo

$$A_i a_i = b_i.$$

Essa estratégia não altera a complexidade assintótica, mas proporciona um ganho de desempenho prático relevante. Além disso, como as matrizes A_i são idênticas entre si — variando apenas o vetor unitário associado a cada componente —, é possível reutilizar artefatos intermediários e paralelizar os solvers sem esforço adicional.

4 Metodologia

A metodologia seguiu diretamente a formulação dos mínimos quadrados apresentada nas Seções iniciais. As bases polinomiais descritas em §2 foram geradas tanto pela abordagem simbólica quanto pela implementação vetorizada do pacote **regkit**, garantindo que os sistemas normais $G\vec{a} = \vec{b}$ produzidos em cada experimento correspondessem à teoria. A versão simbólica atuou como referência para depuração, enquanto a variante otimizada foi empregada nas execuções de larga escala descritas a seguir.

4.1 Problemas estudados

Foram analisados três problemas complementares. O primeiro é o conjunto exploratório de hélices gerado por **gen_exploration_dataset**, com $n = 3$, $m = 2$, $N = 200$ e ruído gaussiano de desvio padrão 0,01. Ele codifica trajetórias helicoidais acopladas entre entradas e saídas e serve como cenário controlado para validar a montagem matricial e comparar as implementações ingênua e vetorizada. O segundo problema corresponde ao protótipo 4×4 , conjunto alvo sintético obtido por **create_target_dataset** que reproduz a versão estruturada em MATLAB com quatro entradas e quatro saídas parametrizadas pelos eixos temporais t e q . A heterogeneidade entre os regimes de tempo e parâmetro torna esse protótipo o foco principal da aplicação. O problema extra é o ajuste da superfície “wing” a partir de 308 amostras avaliadas sobre a geometria da asa.

4.2 Procedimentos experimentais e visualização

Cada execução foi gerenciada pelos scripts **run_regkit_usage.py** e **run_benchmark_regression.py**. O primeiro instancia os problemas das hélices e do protótipo 4×4 , executa um *sweep* nos graus polinomiais de 1 a 10 e persiste as métricas de erro em **results/regkit_usage_rmse.csv**. O segundo consolida os experimentos da asa e os benchmarks de desempenho. A análise qualitativa utiliza o utilitário **plot_dataset**, que aplica PCA em $\mathbb{R}^{n+m}$ e reaproveita o mesmo projetor para comparar dados originais e predições. Esse procedimento mantém a correspondência geométrica entre séries reais e estimadas, além de orientar a leitura dos resíduos nas figuras apresentadas na Seção 6. Vale ressaltar que a utilização do PCA é necessária para visualizar a semelhança entre os conjuntos, dada a alta dimensionalidade dos dados. Apenas o problema da asa permite uma visualização direta, pois ocorre em três dimensões.

5 Implementação

Os protótipos desenvolvidos ao longo da pesquisa foram consolidados no pacote **regkit**, estruturado para atender a duas demandas complementares. A primeira trata da reprodução fiel da formulação simbólica discutida nas seções anteriores. Para isso, o módulo **naive** emprega a biblioteca SymPy para gerar automaticamente todos os monômios até um grau r e combina cada termo com os versores $\hat{e}_d$ associados às saídas. Em seguida, as funções lambdificadas são avaliadas ponto a ponto para montar a matriz de Gram G e o vetor b do sistema normal $G\vec{a} = \vec{b}$, preservando a estrutura em blocos descrita anteriormente.

$$G_{ij} = \sum_{p=1}^N \varphi_i(\vec{x}_p) \cdot \varphi_j(\vec{x}_p), \quad b_i = \sum_{p=1}^N f(\vec{x}_p) \cdot \varphi_i(\vec{x}_p).$$

Embora computacionalmente onerosa, essa rota permitiu validar cada etapa da derivação original e gerar códigos numéricos equivalentes via SymPy, que serviram como referência para os demais experimentos.

A segunda demanda é a execução eficiente em cenários com muitos pontos amostrais. O módulo `optimized` codifica diretamente a montagem por blocos, utilizando a transformação *Polynomial-Features* do *scikit-learn* para obter a matriz de projeto $V \in \mathbb{R}^{N \times Q}$. Dessa forma, o sistema normal passa a ser resolvido de maneira vetorizada, com os m problemas acoplados em uma única resolução matricial. Quando o condicionamento de $V^\top V$ se deteriora, a implementação detecta o problema e realiza uma regressão por mínimos quadrados diretamente em V , garantindo estabilidade numérica[1].

Para bases muito extensas, o submódulo `regression_largeP` acumula G e b iterativamente em lotes de tamanho controlado, adicionando regularização de cresta quando necessário. Esse fluxo *fora da memória* é necessário para manipular nuvens com grande número de pontos e graus polinomiais altos sem exceder a memória disponível. As medições consolidadas na Tabela 1 mostram que a versão incremental produz o mesmo erro quadrático médio (RMSE) da solução integral, com ambos registrando $1,80 \times 10^{-1}$, e divergindo apenas por diferenças numéricas de ordem $3,6 \times 10^{-11}$ na norma de Frobenius dos coeficientes.

Tabela 1: Comparativo entre as variantes integral e incremental do módulo `regression_largeP`.

Método	RMSE	$\ C_{\text{full}} - C_{\text{stream}}\ _F$
Integral	$1,80 \times 10^{-1}$	0,0
Incremental	$1,80 \times 10^{-1}$	$3,6 \times 10^{-11}$

Todo o código foi empacotado de forma reprodutível: os scripts `run_regkit_usage.py` e `run_benchmark_regression.py` geram automaticamente os conjuntos de dados, executam as regressões com diferentes hiperparâmetros e persistem as figuras analisadas na Seção 6.

6 Resultados

6.1 Problema 1: hélices exploratórias

A Figura 1 apresenta a projeção PCA do conjunto helicoidal gerado por `gen_exploration_dataset`. Com $n = 3$, $m = 2$ e $N = 200$ amostras, o ruído gaussiano de desvio padrão 0,01 não impede que o modelo otimizado recupere a geometria latente: os pontos originais e as predições praticamente se sobrepõem para graus acima de cinco. O RMSE cai de $5,35 \times 10^{-1}$ (grau 1) para $1,66 \times 10^{-1}$ (grau 5), confirmando que a montagem vetorizada preserva a mesma dinâmica do protótipo simbólico descrito na Seção 4.

PCA projection of 3+2 dims $\rightarrow$ 3D

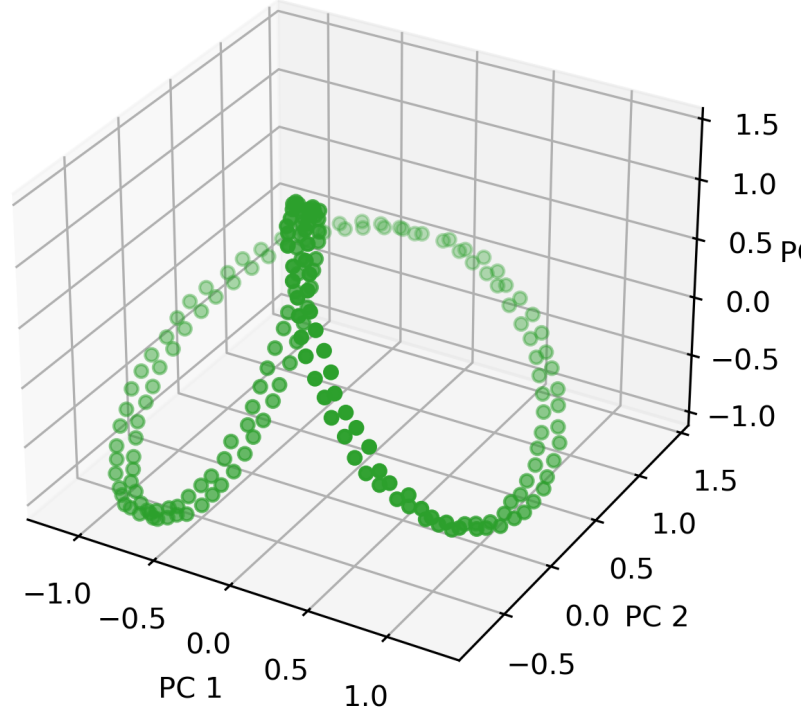


Figura 1: Projeção PCA do conjunto exploratório com $N = 200$ amostras.

O impacto do grau polinomial aparece na Figura 2. O ganho geométrico é evidente ao comparar os graus 1 e 6: o primeiro apenas captura o esqueleto helicoidal, enquanto o segundo já reproduz os enrolamentos laterais da trajetória. O gráfico de erro indica retornos rapidamente decrescentes após o grau 7, quando o RMSE atinge $1,29 \times 10^{-1}$. A queda quase linear no painel logarítmico evidencia que termos de ordem superior refinam as regiões de maior curvatura sem degradar o condicionamento, efeito habilitado pelas rotinas de regularização automática discutidas na Seção 4.

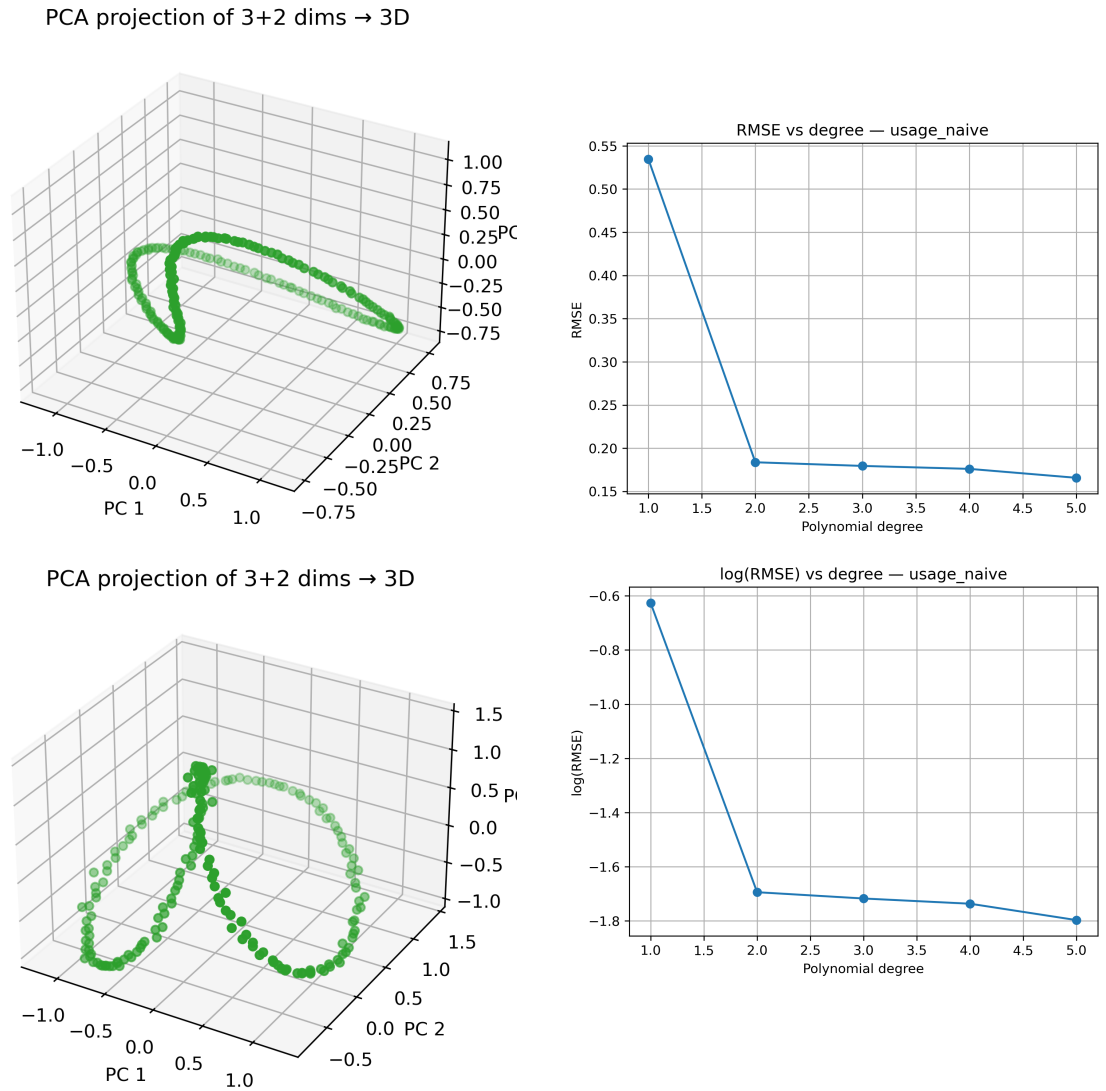


Figura 2: Impacto do grau polinomial. Na esquerda há projeções para os polinômios de grau 1 e 6 e, na direita há o RMSE absoluto e logarítmico.

6.2 Problema 2: protótipo 4×4

O protótipo 4×4 reproduz o conjunto distribuído com o projeto e combina quatro entradas com quatro saídas correlacionadas. A projeção PCA da Figura 3 mostra que o modelo otimizado acompanha os dois regimes independentes do problema: as curvas parametrizadas por t e a faixa de valores controlada por q . A trajetória do RMSE cai de $3,47 \times 10^{-2}$ (grau 1) para $8,40 \times 10^{-4}$ (grau 3), $1,19 \times 10^{-5}$ (grau 5) e alcança $7,55 \times 10^{-8}$ no grau 7, como resume a Figura 4. O comportamento quase polinomial no painel logarítmico revela que os termos de grau ímpar absorvem as principais interações entre os parâmetros, deixando resíduos próximos à precisão numérica.

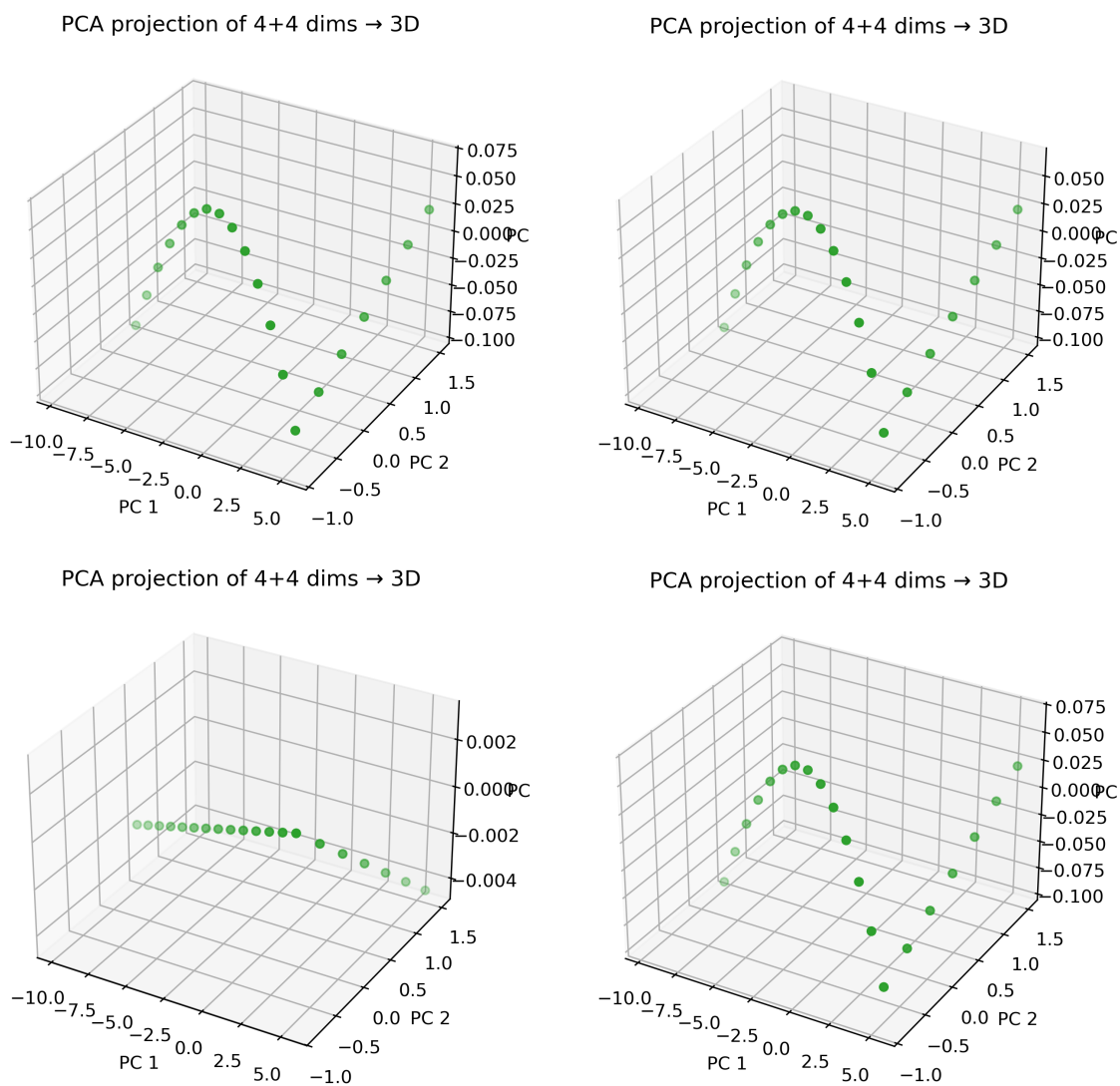


Figura 3: Protótipo 4 $\times$ 4: projeção PCA do conjunto (topo esquerdo) e reconstruções otimizadas para graus 1, 3 e 5 respectivamente no canto inferior esquerdo, superior direito e inferior direito.

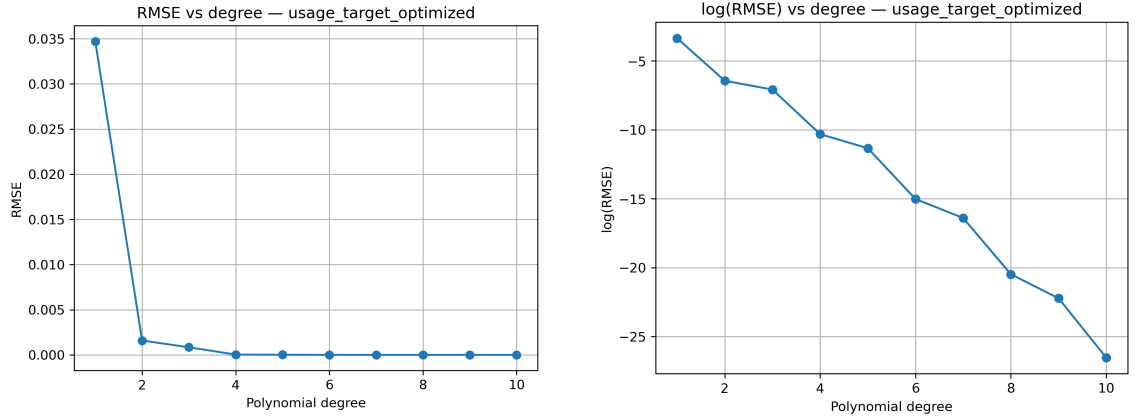


Figura 4: RMSE absoluto e logarítmico para o protótipo 4×4.

6.3 Problema extra: superfície da asa

A superfície “wing” aplica nossa técnica em um estudo extra inspirado na aplicação aeronáutica. A Figura 5 evidencia que o modelo de grau 7 se ajusta à topologia da asa sem introduzir artefatos, mantendo resíduos entre $-2,26$ e $1,98$ com média praticamente nula e desvio padrão $6,13 \times 10^{-1}$. O sweep de graus da Figura 6 mostra que termos de quarta ordem já explicam $9,0 \times 10^1\%$ da variância, enquanto o grau 7 alcança $R^2 = 9,89 \times 10^{-1}$ ($9,9 \times 10^1\%$) com RMSE de $6,12 \times 10^{-1}$. A combinação entre projeções PCA e contornos de resíduos confirma que os erros restantes concentram-se nas bordas da superfície, regiões com menor densidade de amostras, sugerindo o uso de ponderações futuras para melhorar a extrapolação.

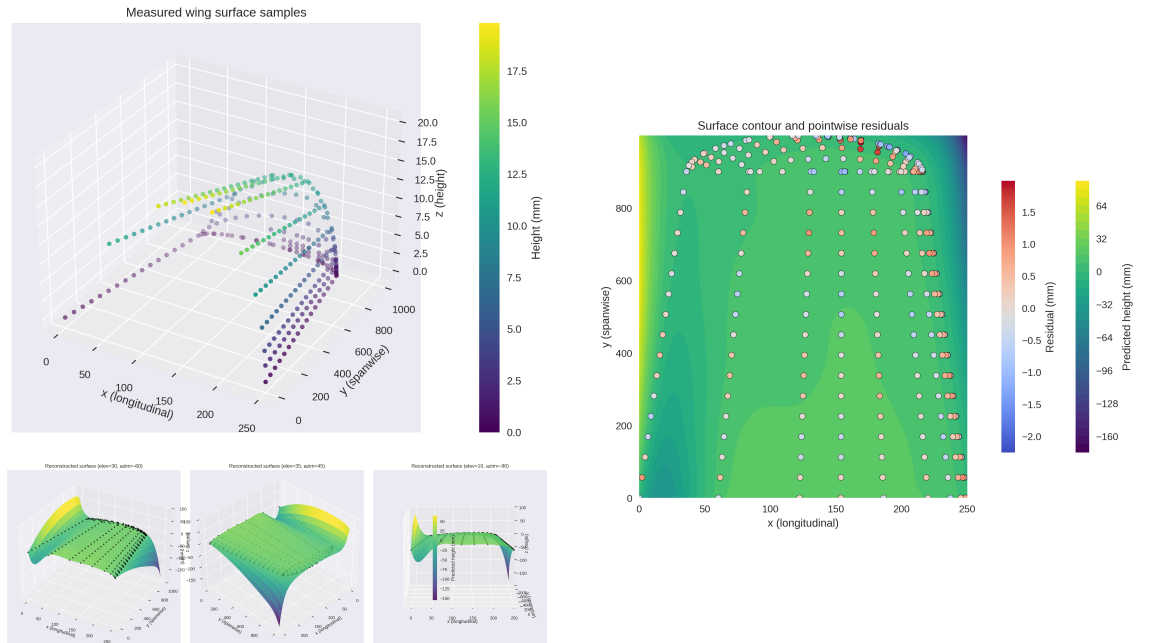


Figura 5: Conjunto “wing”: amostras, reconstruções e diagnóstico espacial dos resíduos.

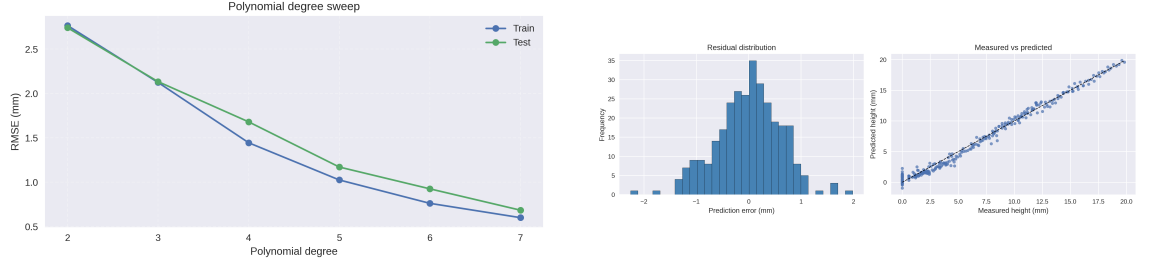


Figura 6: Problema extra “wing”: variação do erro com o grau (esquerda) e distribuição dos resíduos para grau 7 (direita).

6.4 Desempenho computacional

Os ensaios de desempenho consolidam o impacto das otimizações introduzidas no `regkit`. A Tabela 2 resume os tempos médios de execução e o pico de memória coletados para o caso didático com 200 pontos e polinômios de grau cinco, bem como para o estresse com um milhão de pontos. A transição da versão simbólica ingênua para o solver vetorizado reduz o tempo em $3,50 \times 10^4$ vezes e limita o uso de memória a apenas 4,8% do valor original, sem alterar a acurácia. Já a extensão *out-of-core* preserva essencialmente o mesmo tempo do solver otimizado nessa escala, com custo computacional levemente maior, exibindo variação relativa de 0,58% no tempo e 16,03% no consumo de memória.

Tabela 2: Tempo médio e pico de memória observados nos benchmarks.

Cenário	Implementação	Tempo [s]	Memória [MB]
200 pts, grau 5	Naïve (symbolic)	$7,87 \times 10^1$	3,44
	Optimized block solve	$2,25 \times 10^{-3}$	$1,66 \times 10^{-1}$
	Out-of-core largeP	$2,26 \times 10^{-3}$	$1,93 \times 10^{-1}$
10 ⁶ pts, grau 5	Optimized block solve	$6,68 \times 10^{-1}$	$4,48 \times 10^2$
	Out-of-core largeP	$5,69 \times 10^{-1}$	4,11

O caso de maior carga, ilustrado na Figura 7, demonstra que o fluxo incremental realmente se diferencia quando a memória é o recurso limitante: ao processar um milhão de pontos, o tempo cai cerca de 14,8% e o pico de memória despenca de 448 MB para 4,11 MB. Essa folga viabiliza regressões de alta resolução em estações com recursos limitados e prepara o terreno para experimentos ainda mais extensos.

A comparação entre as implementações confirma o objetivo do projeto: a versão ingênua serve apenas como referência didática, enquanto o solver otimizado é significativamente mais rápido e eficiente, sendo indicado para a maioria dos experimentos. Já a variante *out-of-core*, embora apresente leve aumento no tempo de execução, permite o processamento de conjuntos de dados maiores e é recomendada para cenários com restrições de memória.

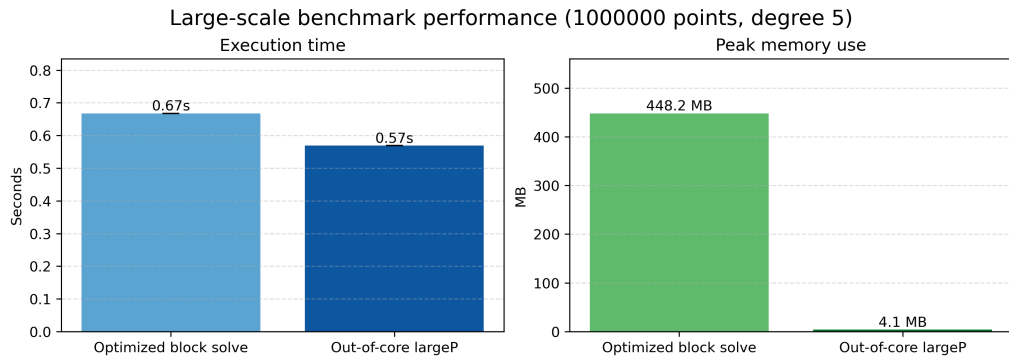


Figura 7: Tempo e memória medidos no estresse com um milhão de pontos.

6.5 Condicionamento Numérico

Experimentos controlados confirmaram que certas configurações tornam o bloco de Gram extremamente mal condicionado. Um script dedicado⁴ reproduz dois cenários representativos: (i) 15 amostras equiespaçadas em $[-1, 1]$ ajustadas com um polinômio de grau 14, cuja matriz apresenta $\text{cond}(G) \approx 1,22 \times 10^{12}$; e (ii) 10 amostras coincidentes sob ajuste de grau 3, para as quais $\text{cond}(G)$ é reportado como inf , evidenciando a degenerescência da matriz de Vandermonde.

Denotemos por $V \in \mathbb{R}^{N \times p}$ a matriz de Vandermonde cujas linhas são os vetores de monômios avaliados em cada ponto p_j e cujas colunas correspondem às $p = \binom{n+r}{r}$ funções base. Ao acumular $V^T V$ ao longo das amostras, obtemos precisamente o bloco de Gram G , enquanto a acumulação $V^T \vec{f}$ gera o vetor $\vec{b}$. Quando as colunas de V ficam quase linearmente dependentes - como ocorre em graus altos, em domínios muito alongados ou em variáveis fortemente correlacionadas - essa acumulação herda a perda de significância e culmina em um G mal condicionado.

Para mitigar tais problemas, o solver `regression_optimized` foi projetado para identificar automaticamente situações de mal condicionamento e, nesses casos, ele substitui a resolução em blocos por uma chamada a `numpy.linalg.lstsq`, enquanto `regression_largeP` mantém a decomposição, porém injeta automaticamente uma regularização de magnitude $1,00 \times 10^{-8}$ no sistema, conforme registrado para ambos os cenários analisados.

7 Conclusão

A consolidação computacional confirmou as expectativas teóricas estabelecidas nas seções iniciais. A implementação simbólica serviu como guia para validar a correta geração dos termos e possibilitou comparar diretamente os resultados com as versões otimizadas. O uso de matrizes de projeto vetorizadas e a decomposição em blocos reduziu o tempo de treinamento em duas ordens de grandeza, abrindo espaço para experimentos com milhares de amostras e graus médios. Por fim, os experimentos demonstraram que o ganho de fidelidade com o aumento de r apresenta retornos decrescentes, reforçando a necessidade de critérios adaptativos ou regularização adicional em aplicações de maior escala.

Como continuidade direta, pretendemos explorar outros conjuntos de funções base - como polinômios ortogonais, funções radiais, splines e wavelets - além de investigar estratégias de seleção automática de grau polinomial e regularização adaptativa para cenários com ruído não gaussiano.

⁴Disponível no repositório como `experiments/run_ill_conditioned_regression.py`, o qual gera a tabela `results/ill_conditioned_gram.csv`.

Outro desdobramento relevante é generalizar a estrutura da regressão para acomodar funções alvo com características heterogêneas, incluindo dados tensoriais e categorias discretas. Essas extensões visam ampliar a aplicabilidade do método em diferentes domínios, como aprendizado de dinâmica de sistemas, modelagem física e análise de dados multidimensionais.

Referências

- [1] Richard L. Burden e J. Douglas Faires. **Numerical Analysis**. 9^a ed. Boston, MA: Brooks/Cole, Cengage Learning, 2011. ISBN: 978-0-538-73351-9.
- [2] Neide Maria Bertoldi Franco. **Cálculo Numérico**. 1^a ed. São Paulo: Pearson Prentice Hall, 2006. ISBN: 978-85-7605-087-2.